The Ouroboros Hardened Framework: Argon2id + XChaCha20-Poly1305 for Offline-Resistant Terminal Authentication

Kevin Thomas
School of Computing and Information
University of Pittsburgh, Pittsburgh, PA, USA
ket189@pitt.edu

Abstract — Password-authenticated systems that embed encrypted artifacts in firmware face a fundamental tension: the key-derivation function must be costly enough to impede offline brute force, yet the artifact-generation tooling must be reproducible so that deployment operators can audit exactly what the firmware will accept. Existing memory-hard KDF + AEAD combinations typically decouple the generator from the runtime, making it difficult to verify that a given encrypted artifact corresponds to a known passphrase without running the full pipeline. We present the Ouroboros Hardened Framework, which closes this gap with a deterministic, implementation-aligned workflow linking passphrase policy, Argon2id parameterization, XChaCha20-Poly1305 authenticated encryption, and a strict 12-word lowercase Diceware-style passphrase policy into a single reproducible artifact contract. The framework generates a JSON artifact that can be cross-compiled into a C header consumed by bare-metal firmware or loaded by a Rust no_std runtime — enabling byte-for-byte verification that generator and decryption path agree. We provide explicit notation, equation-level construction, quantitative classical and optimistic Grover-style brute-force cost analysis, and a discussion of the practical engineering constraints (Argon2id oracle realization cost available quantum computers) that the Grover model omits. All reference implementations are open-source under the MIT license.

Index Terms — Argon2id, XChaCha20-Poly1305, memory-hard KDF, offline guessing resistance, password entropy, Grover scaling, authenticated encryption, Diceware.

I. Introduction

Embedded authentication systems that rely on a single user secret face a well-known tension: the key-derivation function must be expensive enough to thwart offline brute force, yet the encrypted artifact the firmware consumes must be generated by tooling the operator trusts. When generator and runtime are developed independently, subtle parameter drift — a salt length change, a nonce encoding convention, a KDF iteration count that differs between generator and verifier — can produce a system that is either insecure or non-functional without either party noticing.

The hardened Ouroboros framework targets a practical threat boundary: an attacker who captures encrypted artifacts and performs unrestricted offline verification attempts. In this regime, security is determined by two coupled quantities: effective passphrase entropy, and per-guess computational and memory cost. The design objective is to maximize expected attacker work without introducing ambiguous runtime behavior.

The key insight is to make the artifact contract — the format binding generator to firmware — a first-class cryptographic concern, not an implementation detail. By enforcing a strict 12-word Diceware-style passphrase policy, encoding KDF parameters directly into the artifact alongside the ciphertext and nonce, and providing a deterministic round-trip (given fixed inputs, the same ciphertext is always produced), the framework enables byte-for-byte verification that what the generator encrypts is exactly what the firmware will accept.

I.I. Contributions

This paper provides the following concrete contributions:

- A fully specified hardened construction using Argon2id and XChaCha20-Poly1305, with deterministic artifact generation.
- A repository-aligned artifact contract (`demo_artifact.json` / `demo_artifact.h`) with fixed field semantics, enabling cross-implementation verification.
- A mathematical derivation of expected classical crack time and optimistic Grover-style scaling, with discussion of the practical constraints on realizing a Grover oracle for a memory-hard KDF.
- A parameterized interpretation framework that operators can recalculate for hardware-specific calibration (desktop vs embedded profiles).
- Open-source reference implementations in Rust (`no_std`, `#![forbid(unsafe_code)]`) and C (RP2350 bare-metal firmware) under MIT license.

II. Related Work

Password-hardening systems derive security from both secret entropy and asymmetric verification cost. Numerous constructions precede this work.

Bcrypt [1] introduced a tunable iteration count with Blowfish-based key scheduling, but its memory footprint (4 KB) is too small to impede GPU-parallelized attacks. Scrypt [2] added memory hardness with a large pseudorandom array and sequential access pattern, but its parameterization is often deployment-specific and inconsistently applied. Argon2id [3], the winner of the Password Hashing Competition (2015) and standardized in RFC 9106, combines data-dependent memory access with side-channel resistant memory filling and is the KDF adopted in this work.

Authenticated encryption consolidates confidentiality and integrity into a single primitive and avoids ad hoc MAC composition mistakes. XChaCha20- Poly1305 [4, 5] extends the standard ChaCha20-Poly1305 AEAD (RFC 8439) with a 192-bit nonce, eliminating the nonce-reuse concern that shorter-nonce constructions introduce when random nonces are used. It is widely deployed in WireGuard, WhatsApp, and Google's Tink library.

The Ouroboros lineage began with a pure-AVR-assembly authentication engine [6] using Speck-128/256 and a Davies-Meyer key stretch — a hardware-bound construction that imposed an 11-second wall-clock delay per attempt on the ATmega328P. The present hardened framework is a separate, algorithmically distinct construction targeting general-purpose microcontrollers with sufficient SRAM for Argon2id, and prioritizes portable memory-hard KDF cost over hardware-bound rate limiting.

Other embedded authentication frameworks include TPM-backed PCR sealing (which requires a dedicated security chip) and ARM TrustZone-M-based isolated execution (architecture-dependent). The Ouroboros approach relies on no hardware security module beyond the microcontroller's standard peripherals.

III. Formal Model

III.I. Notation

Let:

- P be the user passphrase (policy-constrained to 12 lowercase ASCII words),
- $S \in \{0,1\}^{\{128\}}$ be a per-ciphertext salt,
- $N \in \{0,1\}^{\{192\}}$ be a per-ciphertext nonce,
- $M \in \{0,1\}^{\{384\}}$ be the 48-byte payload,
- (m, t, p) be the Argon2id memory (KiB), time (iterations), and parallelism (lanes) parameters,
- $K_h \in \{0,1\}^{\{256\}}$ be the derived key,
- C be the full ciphertext-plus-authentication-tag output.

The policy accepts only passphrases composed of exactly 12 lowercase ASCII words (each in `[a-z]+`) under ASCII whitespace normalization. This policy is enforced by both generator and firmware, preventing misconfiguration where a weak passphrase is accepted by one but not the other.

III.II. Hardened Construction

Key derivation:

$$K_h = \text{Argon2id}(P, S, m, t, p, 32)$$

Authenticated encryption:

$$C = \text{XChaCha20-Poly1305.Encrypt}(K_h, N, \varepsilon, M)$$

Authenticated decryption:

$$M = \text{XChaCha20-Poly1305.Decrypt}(K_h, N, \varepsilon, C)$$

where ε denotes the empty associated data string. Decryption returns $\perp$ (failure) on authentication tag mismatch. The 48-byte payload M is then dispatched: byte 0 controls an LED, bytes 1–7 are transmitted verbatim over UART, and the remaining bytes are reserved.

III.III. Deterministic Round-Trip Property

For fixed inputs (P, S, N, m, t, p), the encryption output C is deterministic. Therefore the artifact can be regenerated and compared byte-for-byte against the committed firmware header at build time. The RP2350 CMake build system enforces this check, rejecting builds where the generated header is stale relative to the source JSON artifact. This property is operationally useful: documentation examples remain auditable, CI can validate example artifacts, and accidental configuration drift is detectable before deployment.

III.IV. Deployment Parameter Profiles

The framework defines two parameter profiles, reflecting the different memory budgets of desktop and embedded targets.

Parameter	Desktop	Embedded (RP2350)	User range
m (KiB)	1,048,576	64	8–1,048,576
t (passes)	3	3	1–10
p (lanes)	1	1	1–4
Salt	16 B random	16 B random	16 B fixed
Nonce	24 B random	24 B random	24 B fixed

Fig. 1: Default parameter profiles. The embedded profile is constrained by the RP2350's 264 KB total SRAM; the desktop profile requires 1 GB.

The embedded profile (64 KiB, 3 passes, 1 lane) is calibrated for the RP2350 microcontroller's 264 KB SRAM budget. This is an implementation constraint of the target hardware, not a change in the cryptographic construction. The desktop profile (1 GiB, 3 passes, 1 lane) provides the reference cost magnitudes cited in the security analysis below.

IV. Artifact Contract

The runtime demo consumes a JSON blob with the following fields:

```
{
  "format": "ouroboros-hardened-demo-v1",
  "memory_kib": 1048576,
  "iterations": 3,
  "parallelism": 1,
  "salt_hex": "...32 hex chars...",
  "nonce_hex": "...48 hex chars...",
  "ciphertext_and_tag_hex": "...128 hex chars..."
}
```

Field lengths are strict:

- `salt_hex` is exactly 32 hex characters (16 bytes),
- `nonce_hex` is exactly 48 hex characters (24 bytes),
- `ciphertext_and_tag_hex` is exactly 128 hex characters (64 bytes),

with parse-time rejection on malformed or wrong-size input. The C firmware header (`demo_artifact.h`) embeds the same data as static byte arrays and is generated from the JSON artifact by `scripts/dec.py`. A build-time guard fails if the committed header is stale.

V. Payload and Dispatch Semantics

The plaintext payload is fixed at 48 bytes.

Region	Meaning
Byte 0	LED state
Bytes 1..7	UART text bytes
Remaining bytes	Reserved (fixed dispatch)

Fig. 2: 48-byte hardened payload layout.

By default, generation appends CRLF to the user text before packing into bytes 1..7; therefore application text length is constrained by:

$$|\texttt{text}| + 2 \leq 7$$

under default CRLF mode. The `--no-crlf` flag relaxes this to $|\texttt{text}| \leq 7$.

VI. Threat Model and Security Objective

VI.I. Adversary Model

Assume adversary access to:

- Full artifact JSON content (salt, nonce, ciphertext, tag, KDF parameters).
- Complete source code and binary behavior of generator and firmware.
- Unlimited offline trial capability on attacker-controlled hardware.
- No side-channel leakage from trusted runtime outside modeled interfaces.
- No active firmware patching, instruction-level execution control, or injected control-flow faults in the trusted runtime.

These assumptions define an **offline capture** threat model: the attacker has obtained an encrypted artifact and can attempt unlimited guesses but has not compromised the device at the moment of authentication.

VI.II. Security Objective

Delay successful recovery of the passphrase P by maximizing expected offline cost:

$$E[T] = \frac{E[G]}{r}$$

with throughput r reduced by the memory-hard KDF cost per guess, and expected guess count $E[G]$ driven by passphrase entropy.

Notably, the framework does not claim security against:

- Physical UART bus tapping during passphrase entry (the passphrase is transmitted in cleartext over serial),
- Side-channel leakage (power, EM, timing) on the authenticating device,
- Fault injection (glitching, voltage tampering),
- Firmware patching or debug interface bypass.

These exclusions are explicit. Operators deploying the framework in hostile physical environments should add corresponding countermeasures.

VII. Classical Cost Derivation

Let H be effective entropy in bits for the passphrase distribution. Assuming uniform random ordering over 2^H candidates (an optimistic assumption for the attacker):

$$E[G] = \frac{1 + 2^H}{2} \approx 2^{\{H-1\}}$$

where G is the guess index of the first success.

Let r be the attacker's throughput in guesses per second. Then:

$$E[T_{\mathrm{c}}] = \frac{E[G]}{r} \approx \frac{2^{\{H-1\}}}{r}$$

Converting seconds to years ($Y = 31,557,600$):

$$T_{\mathrm{c}} = \frac{2^{\{H-1\}}}{r \cdot Y}$$

VII.I. Per-Guess Cost

The attacker must evaluate Argon2id at the target parameters for each guess. On a high-end GPU (e.g., an RTX 4090), a single Argon2id evaluation at the desktop profile ($m = 1,048,576$, $t = 3$, $p = 1$) takes approximately 0.3–0.5 s — far slower than the Speck-based construction in the original Ouroboros. For the embedded profile ($m = 64$, $t = 3$, $p = 1$), the per-guess time on a GPU is approximately 0.1–0.3 s.

The paper-standard reference scenario therefore uses $r = 0.1$ guesses/s for the desktop profile. This is a **conservative reference estimate**, not a measured benchmark. Deployments must measure their own r under the specific KDF parameterization and expected attacker hardware.

VII.II. Entropy Sensitivity

A one-bit entropy increase doubles expected crack time:

$$T_{\mathrm{c}(H+1)} = 2 \cdot T_{\mathrm{c}(H)}$$

A ten-bit increase multiplies by $2^{\{10\}} = 1024$.

Thus passphrase policy quality is exponentially more important than micro-optimizing any single arithmetic constant in the cost estimator. For this reason, the framework enforces the strictest feasible policy (12 Diceware words) at the generator and firmware level, rather than leaving entropy quality to operator discretion.

VIII. Optimistic Grover-Style Interpretation

A common coarse model for quantum search treats the effective search exponent as halved. Under this optimistic attacker model:

$$E\big[G_{\mathrm{q}}\big] \approx 2^{\{\frac{H}{2}-1\}}$$

With oracle rate r_{q} evaluations per second:

$$T_{\mathrm{q}} = \frac{2^{\{\frac{H}{2}-1\}}}{r_{\mathrm{q}} \cdot Y}$$

VIII.I. Why This Model Is Actually Optimistic for the Attacker

The model above is standard in password-cracking literature as a first-order approximation, but it omits several critical engineering constraints that make a real Grover attack on a memory-hard KDF far more difficult:

1. **Grover requires a quantum oracle for Argon2id.** The oracle must evaluate Argon2id in superposition — including its data-dependent memory access pattern. Memory-hard KDFs are specifically designed to resist parallelization; constructing a Grover oracle that faithfully implements Argon2id's sequential memory access in a fault-tolerant quantum circuit would require an enormous number of logical qubits and T-gates, far beyond any near-term or medium-term quantum architecture [7].
2. **The oracle must be reversible.** Grover's algorithm requires a unitary oracle U_f that computes $f(x) \mapsto \delta_{\{f(x)=y\}}$. Argon2id is not naturally unitary — it mutates a large internal state array with side effects. Each memory access must be uncomputed, doubling the gate count. The resulting T-gate depth for a single oracle call would likely exceed $10^{\{12\}}$ logical gates [7, 8].
3. **Grover is not parallelizable.** Running k copies of Grover's algorithm on k quantum processors reduces the circuit depth by at most a factor of $\texttt{sqrt}\ \{k\}$, and each copy must itself implement the full oracle. The conventional $\texttt{sqrt}\ \{N\}$ speedup is therefore an upper bound that is unlikely to be approached in practice for oracles of this complexity [9].
4. **Fault-tolerant overhead.** The oracle gates must be realized with error-corrected logical qubits. Current estimates for a single logical T-gate on a surface code require $O(10^3)$ physical qubits and $O(10^2)$ physical gate operations [10]. Scaling this to the $10^{\{12\}}$-gate oracle would require physical qubit counts and coherence times that are not projected to be feasible on any known roadmap.

These considerations are included to provide appropriate context for the Grover-style numbers below. The framework makes no claim of formal post-quantum security in the NIST PQC sense. The numbers serve as directional upper bounds on attacker capability, not as proof of irrecoverability under all quantum models.

IX. Worked Scenario and Expanded Math

Set the reference row:

- $H = 155.1$ bits (12-word Diceware: $\log_2\big(7776^{\{12\}}\big) \approx 155.1$),
- $r = 0.1$ guesses/s (conservative desktop-profile reference),
- $r_{\mathrm{q}} = 0.1$ oracle evaluations/s (optimistic quantum oracle rate, assumed equal to classical for comparison — see Section VIII for why this is unrealistic in practice),
- $Y = 31,557,600$ s/year.

Classical expected time:

$$T_{\mathrm{c}} = \frac{2^{\{154.1\}}}{0.1 \times 31,557,600} \approx 7.76 \times 10^{\{39\}} \text{ years}$$

Optimistic quantum expected time:

$$T_{\mathrm{q}} = \frac{2^{\{76.55\}}}{0.1 \times 31,557,600} \approx 3.51 \times 10^{\{16\}} \text{ years}$$

Relative to universe age $U = 1.38 \times 10^{\{10\}}$ years:

$$R_{\mathrm{c}} = \frac{T_{\mathrm{c}}}{U} \approx 5.6 \times 10^{\{29\}}$$

$$R_{\mathrm{q}} = \frac{T_{\mathrm{q}}}{U} \approx 2.5 \times 10^{\{6\}}$$

Metric	Classical	Optimistic Grover
Expected years	$7.76 \times 10^{\{39\}}$	$3.51 \times 10^{\{16\}}$
Universe-age ratio	$5.6 \times 10^{\{29\}}$	$2.5 \times 10^{\{6\}}$

Fig. 3: Reference attack-cost magnitudes for hardened policy target. Quantum numbers are optimistic upper bounds that ignore oracle realization constraints discussed in Section VIII.

This worked row is a paper-standard reference scenario, not a measured desktop benchmark. Actual deployment claims must use measured passphrase entropy quality and deployment-specific per-guess throughput.

IX.I. Why the Numbers Stay Large

Even with exponent-halving assumptions, the combined effect of high entropy and slow per-guess verification preserves extreme expected timescales.

In log-domain terms:

$$\log_2(T_c) = H - 1 - \log_2(r \times Y)$$

$$\log_2(T_q) = \frac{H}{2} - 1 - \log_2(r_q \times Y)$$

So increasing H continues to dominate over moderate throughput gains. A passphrase policy improvement from 10 to 12 Diceware words adds $\log_2(7776^2) \approx 25.9$ bits, multiplying classical crack time by $2^{\{25.9\}} \approx 6.3 \times 10^{\{7\}}$ — far more than any realistic hardware-speedup factor.

X. Implementation Compliance Mapping

The repository runtime and tooling implement the same hardened boundary:

- **Generator:** `scripts/dec.py` writes hardened JSON artifacts and generates C firmware headers.
- **Rust runtime:** `src/bin/demo.rs` loads and validates artifact fields from JSON; `src/lib.rs` exposes `decrypt_hardened` taking (m, t, p, S, N, C).
- **C firmware:** RP2350 `src/auth.c` consumes `include/demo_artifact.h` and runs the identical Argon2id + XChaCha20-Poly1305 path.

A successful firmware decrypt requires both:

- policy-compliant passphrase syntax,
- exact passphrase match with artifact-generation passphrase.

Therefore, policy validity is necessary but not sufficient for successful authentication — an attacker who knows the policy rules but not the secret still cannot authenticate.

X.I. Failure Modes

Authentication failure occurs when any of the following hold:

- Wrong passphrase for current artifact (tag mismatch),
- Artifact corruption or malformed hex fields,
- KDF parameter drift relative to artifact-generation parameters,
- Policy violation (wrong word count or non-lowercase characters) — caught before any crypto operation.

These are explicit, desirable failures — safe states that produce no output and leave the LED in its default state. No undefined behavior path exists for any input that satisfies the parser.

XI. Comparison to the Original Ouroboros

The original Ouroboros [6] was implemented in pure AVR assembly on the ATmega328P (8 MHz, 2 KB SRAM, 32 KB flash) using Speck-128/256 with a Davies-Meyer key stretch of 24,576 iterations (11 seconds per attempt). Its security properties derived primarily from hardware-enforced rate limiting — the attacker was forced to authenticate on-device at 3 attempts per minute.

The present hardened framework differs in three key respects:

1. **Algorithmic basis:** Argon2id + XChaCha20-Poly1305 replace Speck + Davies-Meyer + CTR + embedded MAC. The hardened construction uses standardized, NIST-recognized primitives with provable AEAD security rather than a custom MAC embedding.
2. **Cost model:** Rather than hardware-enforced rate limiting (which is bypassed if the attacker can simulate the algorithm offline), this framework uses memory-hard KDF cost that survives offline attack — the attacker must pay the full Argon2id evaluation

cost per guess regardless of whether the guess is made on authentic hardware or a GPU farm.

3. **Portability:** The hardened construction runs on any microcontroller with sufficient SRAM (264 KB minimum for the embedded profile), rather than requiring the specific AVR instruction set.

The original Ouroboros remains the stronger choice for ultra-constrained 8-bit targets and for threat models where hardware-bound rate limiting (the 11-second-per-attempt wall clock) is the primary defense. The hardened framework is the stronger choice when the attacker can operate offline with unlimited compute, and when hardware capability supports Argon2id.

XII. Limitations and Future Work

XII.I. Limitations

- This framework is not a NIST PQC KEM/signature system. It does not implement lattice, code-based, or multivariate post-quantum primitives.
- Entropy estimates depend on operational passphrase generation quality. The 12-word policy enforces a minimum of 155 bits, but an operator who reuses words, chooses a non-uniform distribution, or shares the passphrase reduces this.
- Cost formulas are expectation models under uniform-guess assumptions; they are not formal proofs of irrecoverability. An attacker with side-channel information (e.g., partial passphrase leakage) could reduce the effective search space.
- Optimistic Grover estimates are directional and may overstate practical attacker capability in near-term or medium-term quantum systems by many orders of magnitude — see Section VIII.
- Only the Rust reference implementation has been formally verified to forbid unsafe code; the C firmware depends on the mbedtls library for AEAD operations and has not been independently audited.
- The passphrase is transmitted in cleartext over UART. Any attacker with physical bus access or a remote serial-over-IP intermediary can capture it directly. This is the weakest link in the current deployment model.

XII.II. Future Work

- **Measured per-guess throughput across deployment profiles.** Systematic benchmarking of Argon2id throughput on the embedded profile (RP2350, ESP32-S3, STM32H7) and the desktop profile (x86_64, Apple Silicon) would enable operators to substitute measured r values into the cost formulas rather than relying on the paper-standard reference.
- **Multi-artifact rotation scheme.** An extension that allows the firmware to hold N encrypted artifacts and cycle through them, such that compromising one does not reveal the passphrase for past or future artifacts.
- **Key-wrapping extension.** A two-layer scheme where the passphrase derives a wrapping key that decrypts a stored per-device key, which in turn decrypts the payload. This would allow passphrase rotation without re-encrypting the artifact.
- **Side-channel evaluation.** Power analysis and electromagnetic emissions testing of the embedded profile on the RP2350, with and without the Argon2id memory-access patterns visible on external SRAM buses.
- **Fault-injection testing.** Clock glitching and voltage tampering evaluation to determine whether the Argon2id data-dependent memory access pattern is exploitable for key recovery.
- **Formal verification of the C firmware path.** Extending the `#![forbid(unsafe_code)]` guarantee to the C implementation, potentially via CBMC or similar bounded model-checking tools.

XIII. Conclusion

The hardened Ouroboros framework provides a complete, implementation-aligned, mathematically explicit path for passphrase-authenticated decryption under offline capture assumptions. The repository's quantitative model is driven by two measurable factors: passphrase entropy and per-guess KDF cost. By coupling strict 12-word Diceware policy enforcement, memory-hard KDF calibration with explicit deployment profiles, and AEAD verification into a single deterministic artifact workflow, the framework avoids ambiguous success criteria and supports reproducible analysis tied to real deployment parameters.

The framework is not a cryptographic proof system, but a practical, open-source, auditable construction for operators who need to ship hardened firmware today. We invite review, benchmarking, and extension by the embedded security community.

XIV. References

[1] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conf. (USENIX ATC '99)*, Monterey, CA, 1999, pp. 81–91.

[2] C. Percival and S. Josefsson, "The scrypt Password-Based Key Derivation Function," RFC 7914, Internet Engineering Task Force, Aug. 2016.

[3] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: The memory-hard function for password hashing and other applications," in *IEEE European Symp. Security and Privacy Workshops (EuroS&PW)*, 2016, pp. 145–156.

[4] F. Denis and S. Lucks, "XChaCha: eXtended-nonce ChaCha and AEAD constructions," IRTF CFRG, Internet-Draft, 2018.

[5] D. J. Bernstein, "ChaCha, a variant of Salsa20," in *Workshop Record of the State of the Art of Stream Ciphers (SASC 2008)*, 2008, pp. 273–278.

[6] K. Thomas, "The Ouroboros Engine: A bare-metal cryptographic authentication framework in pure AVR assembly," arXiv preprint, 2026.

[7] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th ACM Symp. Theory of Computing (STOC '96)*, 1996, pp. 212–219.

[8] M. Amy, O. Di Matteo, V. Gheorghiu, M. Mosca, A. Parent, and J. Schanck, "Estimating the cost of quantum cryptographic oracles," *Lecture Notes in Computer Science*, vol. 10532, Springer, 2017, pp. 28–47.

[9] M. Mosca, "Cybersecurity in an era with quantum computers: Will we be ready?," *IEEE Security & Privacy*, vol. 16, no. 5, pp. 38–41, 2018.

[10] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, "Surface codes: Towards practical large-scale quantum computation," *Physical Review A*, vol. 86, no. 3, 032324, 2012.

[11] NIST, "FIPS 197: Advanced Encryption Standard," National Institute of Standards and Technology, 2001.

[12] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," RFC 8439, Internet Engineering Task Force, Jun. 2018.

[13] IETF CFRG, "Memory-hard password hashing: Argon2 parameterization considerations," IRTF Crypto Forum Research Group, 2020.

[14] A. Reinhold, "Diceware Passphrase Home Page," https://diceware.dmuth.org/

[15] P. Kocher, J. Jaffe, and B. Jun, "Differential power analysis," in *Advances in Cryptology — CRYPTO '99*, Lecture Notes in Computer Science, vol. 1666, Springer, 1999, pp. 388–397.